



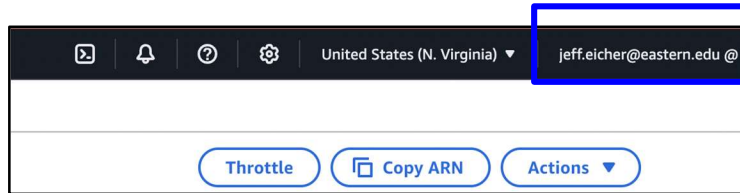
DTSC 671: AI Solutions in the Cloud

SUBMISSION TEMPLATE:

Object Recognition with Amazon Rekognition

Select *File > Make a copy* to edit the template and add your screenshots. Download the completed template as a PDF document to submit to Brightspace.

In every screenshot, include your AWS username that appears in the upper right corner.
Example:



If any one screenshot is missing this identification, the entire assignment will be scored no higher than 70%.

Points totals are provided at the start of each question. You score either full credit or 0 points; there is no partial credit for each question.

Curtis Nielson
DTSC-671

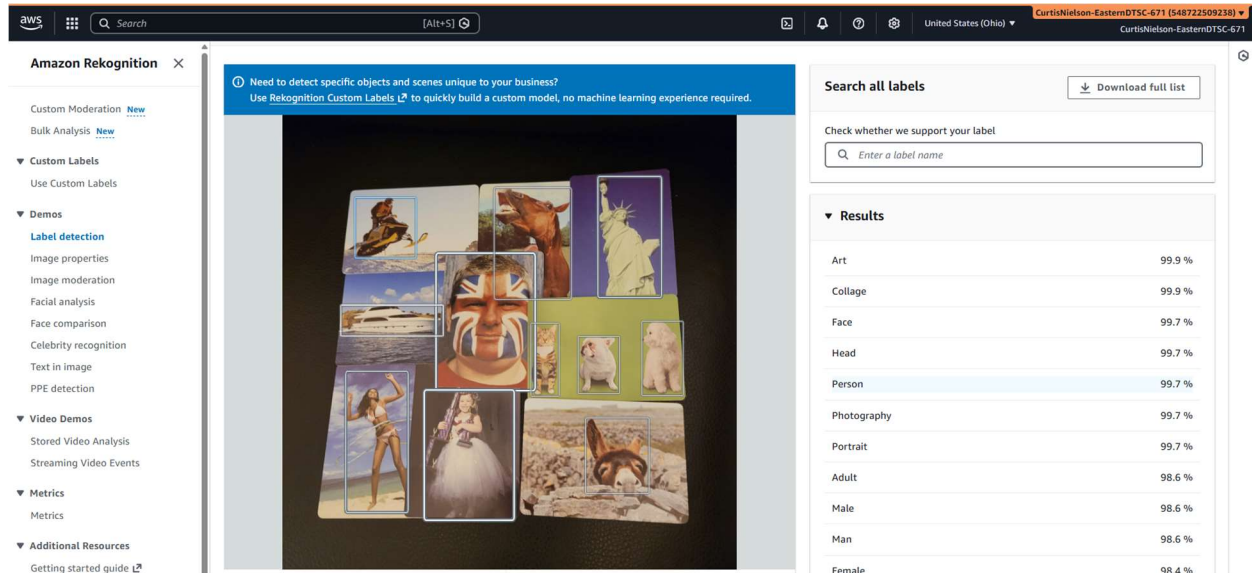
- 1) [5 points] In the space below, provide a screenshot of the image you selected and a description of where you found it (i.e., provide a website if you found the image online; or, if it's your own picture, just let us know). The image should have multiple objects in it (at least three), rather than one single object.



I made a collage of images from the game PicWits and took a picture of the result.

<https://www.mindware.orientaltrading.com/picwits-a2-56002.fltr>

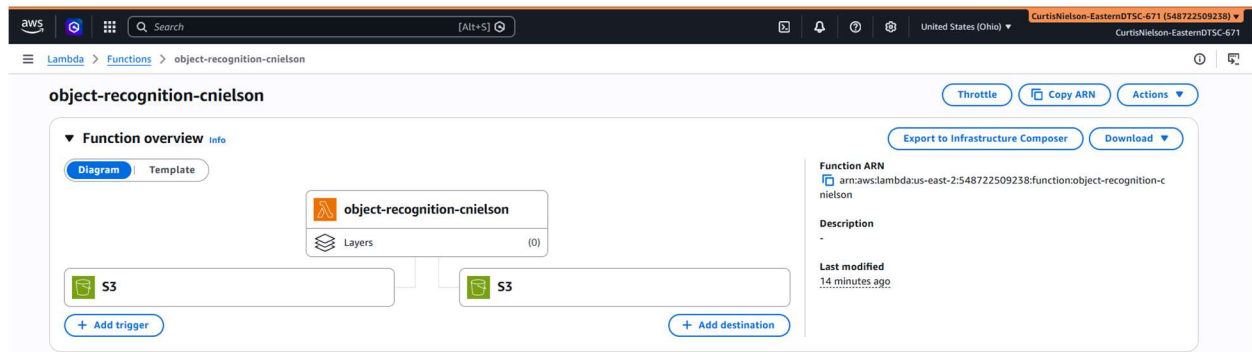
I wanted to give Rekognition a challenge. Here is what it found.



The screenshot shows the Amazon Rekognition console interface. On the left is a navigation menu with options like Custom Moderation, Bulk Analysis, Custom Labels, Demos, Video Demos, Metrics, and Additional Resources. The main area displays a collage of various images including a person on a horse, a statue, a person's face, a car, a person in a bikini, a person in a white dress, and a donkey. To the right of the collage, there is a 'Search all labels' section with a search bar and a 'Download full list' button. Below this is a 'Results' section showing a list of detected labels and their confidence percentages.

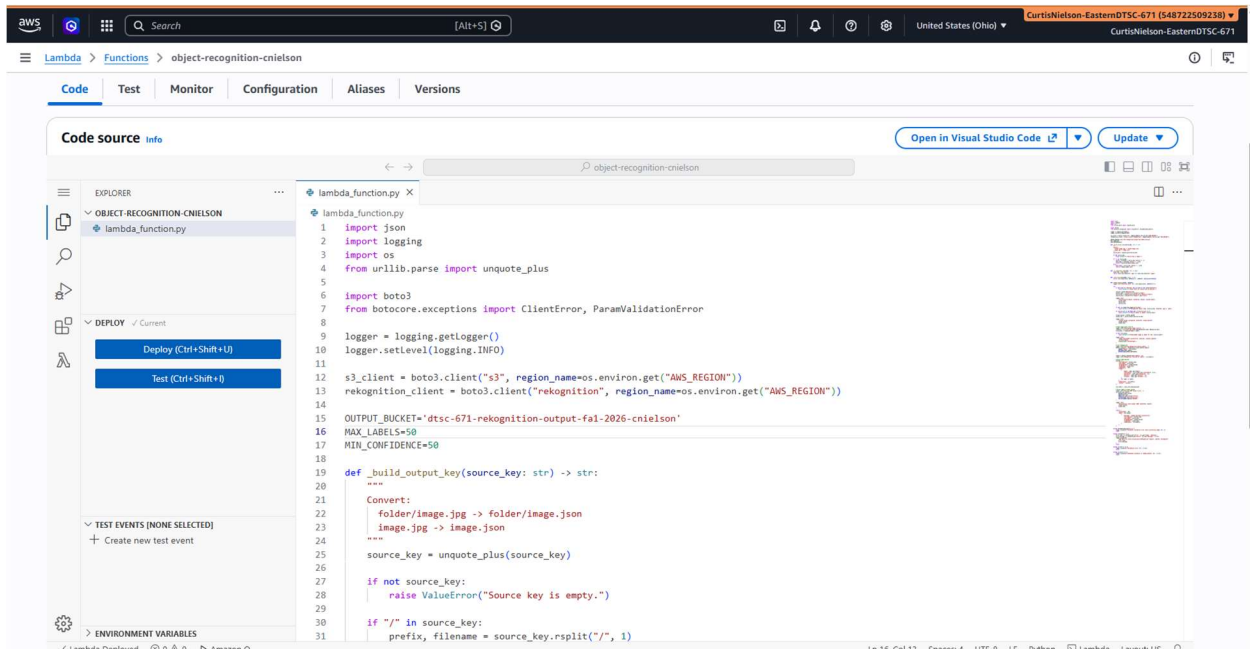
Label	Confidence
Art	99.9 %
Collage	99.9 %
Face	99.7 %
Head	99.7 %
Person	99.7 %
Photography	99.7 %
Portrait	99.7 %
Adult	98.6 %
Male	98.6 %
Man	98.6 %
Female	98.4 %

- 2) [10 points] In the space below, provide a screenshot of the Lambda function diagram (with trigger and destination).



This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

- 3) [10 points] In the space below, provide both a screenshot of the beginning of your Lambda code with your account showing, then copy-paste your full lambda code below the screenshot.



```
import json
import logging
import os
from urllib.parse import unquote_plus
```

```
import boto3
from botocore.exceptions import ClientError, ParamValidationError
```

```
logger = logging.getLogger()
logger.setLevel(logging.INFO)
```

```
s3_client = boto3.client("s3", region_name=os.environ.get("AWS_REGION"))
rekognition_client = boto3.client("rekognition", region_name=os.environ.get("AWS_REGION"))
```

```
OUTPUT_BUCKET='dtsc-671-rekognition-output-fa1-2026-cnielson'
MAX_LABELS=50
MIN_CONFIDENCE=50
```

```
def _build_output_key(source_key: str) -> str:
    """
    Convert:
        folder/image.jpg -> folder/image.json
        image.jpg -> image.json
    """
    source_key = unquote_plus(source_key)
```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

if not source_key:
    raise ValueError("Source key is empty.")

if "/" in source_key:
    prefix, filename = source_key.rsplit("/", 1)
    base_name = filename.rsplit(".", 1)[0]
    return f"{prefix}/{base_name}.json"
else:
    base_name = source_key.rsplit(".", 1)[0]
    return f"{base_name}.json"

def _is_supported_image(key: str) -> bool:
    lower_key = key.lower()
    return lower_key.endswith(".jpg") or lower_key.endswith(".jpeg")

def _safe_json_dump(data: dict) -> str:
    return json.dumps(data, default=str, indent=2, ensure_ascii=False)

def lambda_handler(event, context):
    logger.info("Received event: %s", json.dumps(event, default=str))

    try:
        if not event or "Records" not in event or not event["Records"]:
            raise ValueError("Event does not contain any S3 Records.")

        record = event["Records"][0]
        bucket_name = record["s3"]["bucket"]["name"]
        source_key = unquote_plus(record["s3"]["object"]["key"])
        source_size = record["s3"]["object"].get("size")

        logger.info(
            "Processing S3 object: bucket=%s, key=%s, size=%s bytes",
            bucket_name,
            source_key,
            source_size,
        )

        if not _is_supported_image(source_key):
            raise ValueError(f"Unsupported object type: {source_key}. Expected .jpg or .jpeg")

        if source_size is not None and int(source_size) == 0:
            raise ValueError(f"Source object is empty: {source_key}")

        target_bucket = OUTPUT_BUCKET
        output_key = _build_output_key(source_key)

```

```

logger.info(
    "Target bucket configured: bucket=%s, output_key=%s",
    target_bucket,
    output_key,
)

# Read image bytes from S3
logger.info("Downloading image from S3...")
response = s3_client.get_object(Bucket=bucket_name, Key=source_key)
file_bytes = response["Body"].read()

if not file_bytes:
    raise ValueError(f"Downloaded image is empty for key: {source_key}")

logger.info(
    "Image downloaded successfully. bytes=%s, content_type=%s",
    len(file_bytes),
    response.get("ContentType"),
)

# Call Rekognition
logger.info("Calling Rekognition detect_labels...")
detect_response = rekognition_client.detect_labels(
    Image={"Bytes": file_bytes},
    MaxLabels=MAX_LABELS,
    MinConfidence=MIN_CONFIDENCE,
)

labels = detect_response.get("Labels", [])
logger.info("Rekognition returned %s labels", len(labels))

# Build JSON payload
payload = {
    "sourceBucket": bucket_name,
    "sourceKey": source_key,
    "targetBucket": target_bucket,
    "targetKey": output_key,
    "imageType": "jpg",
    "labels": [
        {
            "Name": label.get("Name"),
            "Confidence": float(label.get("Confidence", 0.0)),
            "Instances": label.get("Instances", []),
            "Parents": label.get("Parents", []),
            "Aliases": label.get("Aliases", []),
        }
        for label in labels
    ],
    "labelCount": len(labels),
    "status": "success",
}

```

```

    }

    json_body = _safe_json_dump(payload)

    # Write JSON to target bucket
    logger.info("Writing JSON output to S3...")
    s3_client.put_object(
        Bucket=target_bucket,
        Key=output_key,
        Body=json_body.encode("utf-8"),
        ContentType="application/json",
        ServerSideEncryption="AES256",
    )

    logger.info(
        "Successfully wrote output JSON: bucket=%s, key=%s",
        target_bucket,
        output_key,
    )

    return {
        "statusCode": 200,
        "body": json.dumps(
            {
                "message": "Image analyzed successfully",
                "sourceBucket": bucket_name,
                "sourceKey": source_key,
                "targetBucket": target_bucket,
                "targetKey": output_key,
                "labelCount": len(labels),
            }
        ),
    }

except ParamValidationError as e:
    logger.exception("Parameter validation error while processing image: %s", e)
    raise

except ClientError as e:
    error_code = e.response.get("Error", {}).get("Code", "Unknown")
    error_message = e.response.get("Error", {}).get("Message", str(e))
    logger.exception(
        "AWS SDK error while processing S3/Rekognition request: code=%s, message=%s",
        error_code,
        error_message,
    )
    raise

except ValueError as e:
    logger.exception("Validation error: %s", str(e))

```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

raise

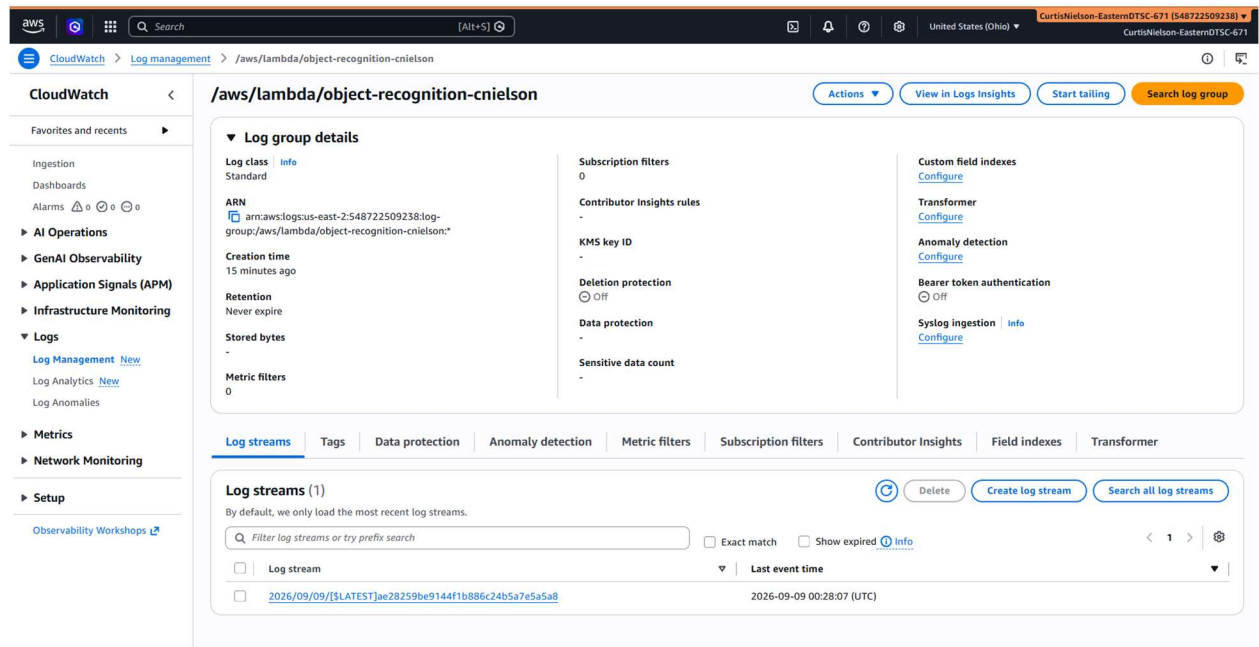
except Exception as e:

```
logger.exception("Unhandled exception in lambda_handler: %s", str(e))
```

raise

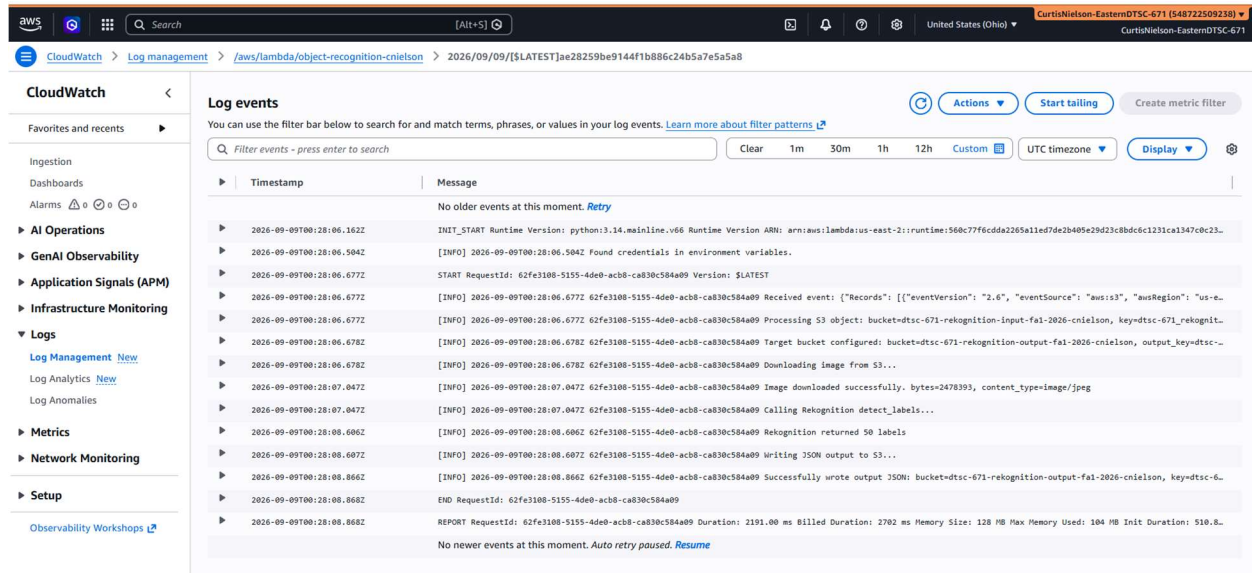
- 4) [10 points] In the space below, provide a screenshot of the CloudWatch log streams for your Lambda function. Fit as many as you can in the screenshot.

I only needed one this time



This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

5) [10 points] In the space below, provide a screenshot of the CloudWatch stream that functioned without errors.



The screenshot displays the AWS CloudWatch 'Log events' console for a Lambda function named `/aws/lambda/object-recognition-cnielson`. The log stream shows the following sequence of events:

- INIT_START** Runtime Version: python3.14.mainline.v66 Runtime Version ARN: arn:aws:lambda:us-east-2:runtime:560c77f6cdda2265e11ed7de2b405e29d23c8bd6c1231ca1347c0c23...
- [INFO]** 2026-09-09T00:28:06.504Z Found credentials in environment variables.
- START** RequestId: 62fe3108-5155-4de0-acb8-ca830c584a09 Version: \$LATEST
- [INFO]** 2026-09-09T00:28:06.677Z Received event: {"Records": [{"eventVersion": "2.6", "eventSource": "aws:s3", "awsRegion": "us-e...}
- [INFO]** 2026-09-09T00:28:06.677Z Processing S3 object: bucket=dtsc-671-rekognition-input-fal-2026-cnielson, key=dtsc-671_rekognit...
- [INFO]** 2026-09-09T00:28:06.678Z Target bucket configured: bucket=dtsc-671-rekognition-output-fal-2026-cnielson, output_key=dtsc...
- [INFO]** 2026-09-09T00:28:06.678Z Downloading image from S3...
- [INFO]** 2026-09-09T00:28:07.047Z Image downloaded successfully. bytes=2478393, content_type=image/jpeg
- [INFO]** 2026-09-09T00:28:07.047Z Calling Rekognition detect_labels...
- [INFO]** 2026-09-09T00:28:08.606Z Rekognition returned 50 labels
- [INFO]** 2026-09-09T00:28:08.607Z Writing JSON output to S3...
- [INFO]** 2026-09-09T00:28:08.866Z Successfully wrote output JSON: bucket=dtsc-671-rekognition-output-fal-2026-cnielson, key=dtsc-6...
- END** RequestId: 62fe3108-5155-4de0-acb8-ca830c584a09
- REPORT** RequestId: 62fe3108-5155-4de0-acb8-ca830c584a09 Duration: 2191.00 ms Billed Duration: 2702 ms Memory Size: 128 MB Max Memory Used: 104 MB Init Duration: 510.8...

The console indicates 'No older events at this moment. [Retry](#)' and 'No newer events at this moment. [Auto retry paused. **Resume**](#)'.

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

- 6) [10 points] In the space below, provide a screenshot of IAM role permissions for the Lambda function.

The top screenshot shows the AWS Lambda console for the function 'object-recognition-cnielson'. The 'Configuration' tab is selected, showing the 'Execution role' as 'object-recognition-cnielson-role-7m7rgk2c'. The 'Resource summary' section lists 'Amazon CloudWatch Logs' as a resource.

The bottom screenshot shows the AWS IAM console for the role 'object-recognition-cnielson-role-7m7rgk2c'. The 'Permissions' tab is selected, showing a list of three attached policies:

Policy name	Type	Attached entities
AmazonRekognitionFullAccess	AWS managed	1
AmazonS3FullAccess	AWS managed	4
AWSLambdaBasicExecutionRole-c916b435-ed93-4d17-a91a-383...	Customer managed	1

- 7) [20 points] In the space below, provide a screenshot of the JSON file in your output bucket.

The screenshot shows the Amazon S3 console for the bucket 'dtsc-671-rekognition-output-fa1-2026-cnielson'. The 'Objects' tab is selected, showing a list of objects. The object 'dtsc-671_rekognition.json' is highlighted.

Name	Type	Last modified	Size	Storage class
dtsc-671_rekognition.json	json	September 8, 2026, 20:28:09 (UTC-04:00)	19.6 KB	Standard

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

- 8) [25 points] Provide a screenshot of your JSON file's output (which shows labels and confidence). Fit as much as you can in the screenshot. We expect this information to match the image you provided above.

```
{
  "sourceBucket": "dtsc-671-rekognition-input-fal-2026-cnielson",
  "sourceKey": "dtsc-671_rekognition.jpg",
  "targetBucket": "dtsc-671-rekognition-output-fal-2026-cnielson",
  "targetKey": "dtsc-671_rekognition.json",
  "imageType": "jpg",
  "labels": [
    {
      "Name": "Art",
      "Confidence": 99.97803497314453,
      "Instances": [],
      "Parents": [],
      "Aliases": []
    },
    {
      "Name": "Collage",
      "Confidence": 99.97803497314453,
      "Instances": [],
      "Parents": [
        {
          "Name": "Art"
        }
      ],
      "Aliases": []
    },
    {
      "Name": "Face",
      "Confidence": 99.7634048461914,
      "Instances": [],
      "Parents": []
    }
  ]
}
```

```
{
  "Name": "Face",
  "Confidence": 99.7634048461914,
  "Instances": [],
  "Parents": [
    {
      "Name": "Head"
    },
    {
      "Name": "Person"
    }
  ],
  "Aliases": []
},
{
  "Name": "Head",
  "Confidence": 99.7634048461914,
  "Instances": [],
  "Parents": [
    {
      "Name": "Person"
    }
  ],
  "Aliases": []
},
{
  "Name": "Person",
  "Confidence": 99.7634048461914,
  "Instances": [],
  "Parents": []
}
```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.